

# Implementing Inline Languages in Python

**Kyle Richardson**

Allen Institute for AI (AI2)

September 2025

# Our simple probabilistic language

- ▶ We presented a small, Python-like probabilistic programming language for prompting. Programs look like:

# Our simple probabilistic language

- ▶ We presented a small, Python-like probabilistic programming language for prompting. Programs look like:

```
1 | P1 =~ Decision(0.86)      ## <-- new syntax
2 | if P1:
3 |     P2 =~ Decision(0.53) ## <-- new syntax
4 |     a = "Paris"
5 |     if P2: return a
6 |     else: raise ValueError(a)
7 | else:
8 |     raise ValueError(P1)
```

*added syntax:* **probabilistic variable assignment** ( $=\sim$ ) and a **Decision** operator for drawing samples from distributions.

# Our simple probabilistic language

- ▶ We presented a small, Python-like probabilistic programming language for prompting. Programs look like:

```
1 | P1 =~ Decision(0.86)      ## <-- new syntax
2 | if P1:
3 |     P2 =~ Decision(0.53) ## <-- new syntax
4 |     a = "Paris"
5 |     if P2: return a
6 |     else: raise ValueError(a)
7 | else:
8 |     raise ValueError(1)
```

This is mostly just ordinary Python

*added syntax:* probabilistic variable assignment ( $=\sim$ ) and a `Decision` operator for drawing samples from distributions.

**Question:** Can we use the native Python interpreter to implement this language (vs. implementing it from scratch)?

# Our simple probabilistic language

- ▶ We presented a small, Python-like probabilistic programming language for prompting. Programs look like:

```
1 | P1 =~ Decision(0.86)      ## <-- new syntax
2 | if P1:
3 |     P2 =~ Decision(0.53) ## <-- new syntax
4 |     a = "Paris"
5 |     if P2: return a
6 |     else: raise ValueError(a)
7 | else:
8 |     raise ValueError(P1)
```

*added syntax:* **probabilistic variable assignment** ( $=\sim$ ) and a **Decision** operator for drawing samples from distributions.

**Question:** Can we use the native Python interpreter to implement this language (vs. implementing it from scratch)?

# Implementing languages via Python's ast module

```
1  import ast
2
3  P = """
4  P1 =~ Decision(0.86)
5  if P1:
6      P2 =~ Decision(0.53)
7      a = "Paris"
8      if P2: return a
9      else: raise ValueError(a)
10 else: raise ValueError(P1)
11 """
12
13 ## get program's abstract syntax tree
14 ## via the Python parser
15 tree = ast.parse(P)
```

- ▶ **ast:** Module system in the standard library, allows for reading Python code as data and making transformations on code.

# Implementing languages via Python's ast module

```
1 import ast
2
3 p = ""
4 # P1: Declaration
5 if P1:
6     P2 = Declaration()
7     a = Param()
8     at P2: return a
9     else: raise ValueError()
10 # P2: Declaration
11
12 # get program & check if it is valid
13 # via the Python parser
14 tree = ast.parse(P)
```

**Idea:** Implement languages via code transformations

- ▶ ast: Module system in the standard library, allows for reading Python code as data and making transformations on code.

# Implementing languages via Python's ast module

```
1  import ast
2
3  P = """
4  P1 =~ Decision(0.86)
5  if P1:
6      P2 =~ Decision(0.53)
7      a = "Paris"
8      if P2: return a
9      else: raise ValueError(a)
10 else: raise ValueError(P1)
11 """
12
13 ## get program's abstract syntax tree
14 ## via the Python parser
15 tree = ast.parse(P)
```

- ▶ ast: Module system in the standard library, allows for reading Python code as data and making transformations on code.

# Implementing languages via Python's ast module

```
1  ## print readable abstract syntax tree
2  print(ast.dump(tree, indent=True))
```

**control structure**

```
2  if P1:
3      P2 ==~ Decision(0.53)
4      a = "Paris"
5      if P2: return a
6      else: raise ValueError(a)
```

**assignment**

1 | P1 ==~ Decision(0.86)

**exception**

6 | else: raise ValueError(a)

```
Module(
  body=[
    Assign(
      targets=[
        Name(id='P1', ctx=Store())],
      value=UnaryOp(
        op=Invert(),
        operand=Call(
          func=Name(id='Decision', ctx=Load()),
          args=[
            Constant(value=0.86)],
          keywords=[])),
      keywords=[]),
    If(
      test=Name(id='P1', ctx=Load()),
      body=[
        Assign(
          targets=[
            Name(id='P2', ctx=Store())],
            value=UnaryOp(
              op=Invert(),
              operand=Call(
                func=Name(id='Decision', ctx=Load()),
                args=[
                  Constant(value=0.53)],
                keywords=[])),
            keywords=[]),
          Raise(
            exc=Call(
              func=Name(id='ValueError', ctx=Load()),
```

# Implementing languages via Python's ast module

```
1  ## print readable abstract syntax tree
2  print(ast.dump(tree,indent=True))
```

control structure

**note:** `=~` is valid Python, but not assignment

assignment

```
1 | P1 =~ Decision(0.86)
```

```
6 | else: raise ValueError(a)
```

exception

# Implement via Python's ast module

```
1  ## return back to raw code
2  print(ast.unparse(tree))
3  ## =>
4  # P1 = ~Decision(0.86)
5  # if P1:
6  #     P2 = ~Decision(0.53) ## <--- bit operator
7  #     a = 'Paris'
8  #     if P2:
9  #         return a
10 #     else:
11 #         raise ValueError(a)
12 # else:
13 #     raise ValueError(P1)
```

► ~ is a unary bit-operator in Python, modifies value in assignment.

## Implement via Python's ast module

```
1  ## return back to raw code
2  print(ast.unparse(tree))
3  ## =>
```

```
Assign(
  targets=[
    Name(id='P1', ctx=Store())],
  value=UnaryOp(
    op=Invert(),
    operand=Call(
      func=Name(id='Decision', ctx=Load()),
      args=[
        Constant(value=0.86)],
      keywords=[])),
```

**invert**

**function**

► ~ is a unary bit-operator in Python, modifies value in assignment.

## Implement via Python's ast module

```
1  ## return back to raw code
2  print(ast.unparse(tree))
3  ## =>
4  # P1 = "Decision(0.86)
5  # if P1:
6  #     P2 = "Decision(0.53) ## <--- bit operator
7  #     a = 'Paris'
8  #     if P2:
9  #         return a
10 #     raise ValueError(a)
11 # else:
12 #     raise ValueError(P1)
```

**Step 1:** Rewrite tree to modify details of  $=\sim$

►  $\sim$  is a unary bit-operator in Python, modifies value in assignment.

# Visiting and transforming via NodeTransformers

```
1 class ProgramTransformer(ast.NodeTransformer):
2     """Transformer underlying abstract syntax tree"""
3
4     def visit_Assign(self, node):
5         """Rewrite our custom assign operator."""
6         if isinstance(node.value, ast.UnaryOp) \
7             and isinstance(node.value.op, ast.Invert):
8             node.value = node.value.operand
9             ## check that =~ only contains a
10            ## decision operator on right.
11            if not (
12                isinstance(node.value, ast.Call) \
13                and node.value.func.id == "Decision"
14            ):
15                raise SyntaxError(
16                    "Incorrect prob. assignment."
17                )
18            return self.generic_visit(node)
```

- ▶ Traverses through syntax tree via visit\_**NodeType**(node) methods, allows for modifying components.

# Visiting and transforming via NodeTransformers

```
1  tree = ProgramTransformer().visit(tree)
2  ## lower level fix of modified code
3  ast.fix_missing_locations(tree)
4  ## print transformed code
5  print(ast.unparse(tree))
6  ## =>
7  # P1 = Decision(0.86)
8  # if P1:
9  #     P2 = Decision(0.53) <-- ordinary assignment
10 #     a = "Paris"
11 #     if P2:
12 #         return a
13 #     else:
14 #         raise ValueError(a)
15 # else:
16 #     raise ValueError(P1)
```

- Transforms our probabilistic assignment into ordinary assignment, checks that it involves the Decision operator.

# Visiting and transforming via NodeTransformers

```
1  tree = ProgramTransformer().visit(tree)
2  ## lower level fix of modified code
3  ast.fix_missing_locations(tree)
4  ## print transformed code
5  print(ast.unparse(tree))
6  ## =>
7  * P1 = Decision(0.86)
8  * if P1:
9  *     P2 = Decision(0.53) <-- ordinary assignment
10 *     a = "Paris"
11 *
12 * else:
13 *     raise ValueError(a)
14 *
15 * else:
16 *     raise ValueError(P1)
```

## Step 2: Define Decision, compile transformed code

- Transforms our probabilistic assignment into ordinary assignment, checks that it involves the Decision operator.

# Implementing our Decision operator

```
1  from numpy.random import choice
2
3  def Decision(prob: float): ## <-- Implementation of Decision operator
4      return choice([True, False], p=[prob, 1 - prob])
5
6  def compile_code(code, fname = 'my_fun'):
7      ## make into definition if not already
8      if "def " not in code:
9          P_lines = ["    "+l for l in P.split('\n')]
10         code = f"def {fname}():{'\n'.join(P_lines)}"
11         tree = ast.parse(code)
12         tree = ProgramTransformer().visit(tree)
13         ast.fix_missing_locations(tree)
14
15         code = compile(tree, mode="exec")
16         locs = {}
17
18         ## execute via exec with Decision in scope
19         exec(code, {"Decision": Decision}, locs)
20         return locs[fname]
```

# Putting it together

```
1  P = """
2  P1 =~ Decision(0.86)
3  if P1:
4      P2 =~ Decision(0.53)
5      a = "Paris"
6      if P2: return a
7      else: raise ValueError(a)
8  else: raise ValueError(P1)
9  """
10
11  ## executable form of custom code
12  my_program = compile_code(P)
13
14  ## execute compiled probabilistic program
15  my_program()
16  # =>
17  # Traceback (most recent call last):
18  #   File ".....", line 55, in <module>
19  #     prog()
20  #   File "<ast>", line 7, in my_fun
21  # ValueError: Paris
```

## Putting it together: **syntax error**

```
1  P = """
2  P1 =~ Decision(0.86)
3  if P1:
4      P2 =~ 10 ## <-- wrong assignment
5      a = "Paris"
6      if P2: return a
7      else: raise ValueError(a)
8  else: raise ValueError(P1)
9  """
10
11  ## executable form of custom code
12  my_program = compile_code(P)
13
14  ## execute compiled probabilistic program
15  my_program()
16  # =>
17  # SyntaxError: Wrong prob. assignment
```

## Putting it together: **syntax error**

```
1  P = """
2  P1 =~ Decision(0.86)
3  if P1:
4      P2 =~ 10 ## <-- wrong assignment
5      a = "Paris"
6      if P2: return a
7      else: raise ValueError(a)
8  else: raise ValueError(P1)
9  """
10
11 ## executable form of custom code
12 my_program = compile_code(P)
13
14 ## execute compiled probabilistic program
15 my_program()
16 # =>
17 # SyntaxError: Wrong prob. assignment
```

- **Exercise:** Implement more syntactic forms (e.g., observe), syntactic checks to restrict the language more.

## Putting it together: **syntax error**

```
1 p = 100
2 p = Decision(0.00)
3 if P1:
4     p = 10 # wrong assignment
5     x = "Paris"
6     if P2: return x
7     else: raise ValueError()
8 else: raise ValueError()
9 return
10
```

Writing our programs *inline* (inside of Python).

```
10 # compile probabilistic program
11 my_program()
12 # =>
13 # SyntaxError: wrong prob. assignment
```

- Exercise: Implement more syntactic forms (e.g., observe), syntactic checks to restrict the language more.

# Inlining code via decorators

```
1  class Compiler:
2      ## definition of decorator
3      def __call__(self, p):
4          pname = p.__name__
5          program_src = inspect.getsource(p)[11:]
6          compiled = compile_code(program_src, pname)
7          @functools.wraps(p)
8          def wrapper(*args,**kwargs):
9              return compiled(*args, **kwargs)
10         return wrapper
11
12  @Compiler() ## <-- decorator, wraps around program, reads as data
13  def my_program():
14      P1 =~ Decision(0.86)
15      if P1:
16          P2 =~ Decision(0.53)
17          a = "Paris"
18          if P2: return a
19          else: raise ValueError(a)
20      else:
21          raise ValueError(P1)
```

# Inlining code via decorators

```
1  @Compiler()
2  def my_program():
3      P1 =~ Decision(0.86)
4      if P1:
5          P2 =~ Decision(0.53)
6          a = "Paris"
7          if P2: return a
8          else: raise ValueError(a)
9      else:
10         raise ValueError(P1)
11
12  ## execute compiled Python program
13  my_program()
14  # => "Paris"
```

► **Advantages:** uses native Python parser, compiles to efficient byte-code.

**Limitations:** must be valid Python syntax.

# The trick (and limitation)

```
1  @Compiler()
2  def my_program():
3      P1 =~ Decision(0.86)
4      if P1:
5          P2 := Decision(0.53) ## <-- Not valid Python
6          a = "Paris"
7          if P2: return a
8          else: raise ValueError(a)
9      else:
10         raise ValueError(P1)
11
12     ### We are limited to Python syntax, in
13     ### order to get access to the Python parser.
```

► **Advantages:** uses native Python parser, compiles to efficient byte-code.

**Limitations:** must be valid Python syntax.

# Example ast-based programming languages

<https://github.com/pcarbons/pyDatalog>

```
@pydatalog.program()
def _():
    # when is it ok to have a queen in row X1 and another in row X2, separated by N columns
    # this is memoized !
    ok(X1, N, X2) <- (X1 != X2) & (X1 != X2+N) & (X1 != X2-N)

    queens(X8)
    queens(X8,X1) <- queens(X8) & next_queen(X8,X1,X1)
    queens(X8,X1,X2) <- queens(X8,X1) & next_queen(X8,X1,X2)
    queens(X8,X1,X2,X3) <- queens(X8,X1,X2) & next_queen(X8,X1,X2,X3)
    queens(X8,X1,X2,X3,X4) <- queens(X8,X1,X2,X3) & next_queen(X8,X1,X2,X3,X4)
    queens(X8,X1,X2,X3,X4,X5) <- queens(X8,X1,X2,X3,X4) & next_queen(X8,X1,X2,X3,X4,X5)
    queens(X8,X1,X2,X3,X4,X5,X6) <- queens(X8,X1,X2,X3,X4,X5) & next_queen(X8,X1,X2,X3,X4,X5,X6)
    queens(X8,X1,X2,X3,X4,X5,X6,X7) <- queens(X8,X1,X2,X3,X4,X5,X6) & next_queen(X8,X1,X2,X3,X4,X5,X6,X7)

    next_queen(X8,X1) <- queens(X1) & ok(X8,1,X1)
    next_queen(X8,X1,X2) <- next_queen(X1,X2) & ok(X8,2,X2)
    next_queen(X8,X1,X2,X3) <- next_queen(X1,X2,X3) & ok(X8,3,X3)
    next_queen(X8,X1,X2,X3,X4) <- next_queen(X1,X2,X3,X4) & ok(X8,4,X4)
    next_queen(X8,X1,X2,X3,X4,X5) <- next_queen(X1,X2,X3,X4,X5) & ok(X8,5,X5)
    next_queen(X8,X1,X2,X3,X4,X5,X6) <- next_queen(X1,X2,X3,X4,X5,X6) & ok(X8,6,X6)
    next_queen(X8,X1,X2,X3,X4,X5,X6,X7) <- next_queen(X1,X2,X3,X4,X5,X6,X7) & ok(X8,7,X7)
```

<https://github.com/eth-sri/lmql>

```
@lmql.query
def chain_of_thought(question):
    """lmql

    # Q&A prompt template
    "Q: {question}\n"
    "A: Let's think step by step.\n"
    "[REASONING]"
    "Thus, the answer is:[ANSWER]."
```

```
# return just the ANSWER to the caller
return ANSWER
"""

print(chain_of_thought("Today is the 12th of June, what day was it 1 week ago?"))
```

<https://github.com/hylang/hy>

## Hy (programming language)

🔗 5 languages ▾

Article Talk

Read Edit View history Tools ▾

From Wikipedia, the free encyclopedia

**Hy** is a *dialect* of the **Lisp** programming language designed to interact with **Python** by translating *s-expressions* into Python's *abstract syntax tree* (AST).<sup>[2][3]</sup> Hy was introduced at **Python Conference** (PyCon) 2013 by Paul Tagliamonte.<sup>[4]</sup> Lisp allows operating on code as data (*metaprogramming*), thus Hy can be used to write *domain-specific languages*.<sup>[5]</sup>

Similar to **Kawa**'s and **Clojure**'s mappings onto the Java virtual machine (JVM),<sup>[6][7]</sup> Hy is meant to operate as a transparent Lisp front-end for Python.<sup>[8]</sup> It allows Python *libraries*, including the *standard library*, to be imported and accessed alongside Hy code with a *compiling*<sup>[note 1]</sup> step where both languages are converted into Python's AST.<sup>[note 2][9][10][11]</sup>

### Hy



Hy logo – Cuddles the cuttlefish

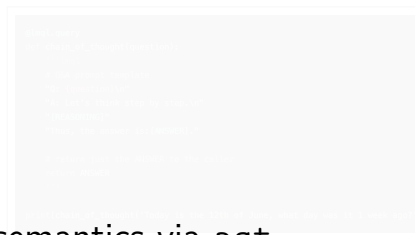
|                    |                                                                                    |
|--------------------|------------------------------------------------------------------------------------|
| <b>Paradigm</b>    | Multi-paradigm: procedural, functional, object-oriented, meta, reflective, generic |
| <b>Family</b>      | Lisp                                                                               |
| <b>Designed by</b> | Paul Tagliamonte                                                                   |

# Example ast-based programming languages

<https://github.com/pccarbons/pyDataLog>

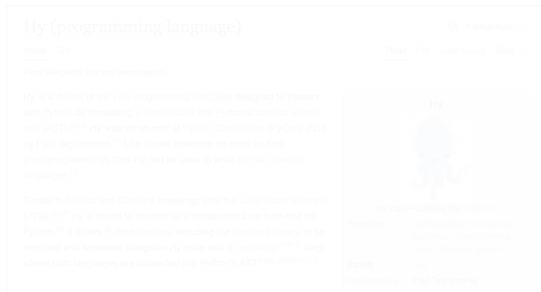


<https://github.com/eth-ari/Inql>



## Program analysis and semantics via ast

<https://github.com/hy-lang/hy>



# Translating programs into weighted logic

Programming  
Languages

T.A. Standish  
Editor

## Guarded Commands, Nondeterminacy and Formal Derivation of Programs

Edsger W. Dijkstra  
Burroughs Corporation

---

So-called “guarded commands” are introduced as a building block for alternative and repetitive constructs that allow nondeterministic program components for which at least the activity evoked, but possibly even the final state, is not necessarily uniquely determined by the initial state. For the formal derivation of programs expressed in terms of these constructs, a calculus will be shown.

**Key Words and Phrases:** programming languages, sequencing primitives, program semantics, programming language semantics, nondeterminacy, case-construction, repetition, termination, correctness proof, derivation of programs, programming methodology

**CR Categories:** 4.20, 4.22

### 1. Introduction

In Section 2, two statements, an alternative construct and a repetitive construct, are introduced, together with an intuitive (mechanistic) definition of their semantics. The basic building block for both of them is the so-called “guarded command,” a statement list prefixed by a boolean expression: only when this boolean expression is initially true, is the statement list eligible for execution. The potential nondeterminacy allows us to map otherwise (trivially) different programs on the same program text, a circumstance that seems largely responsible for the fact that programs can now be derived in a manner more systematic than before.

In Section 3, after a prelude defining the notation, a formal definition of the semantics of the two constructs is given, together with two theorems for each of the constructs (without proof).

In Section 4, it is shown how, based upon the above, a formal calculus for the derivation of programs can be founded. We would like to stress that we do not present “an algorithm” for the derivation of programs: we have used the term “a calculus” for a formal discipline—a set of rules—such that, if applied successfully: (1) it will have derived a correct program; and (2) it will tell us that we have reached such a goal. (We use the term as in “integral calculus.”)

- We defined a simple semantics for our language in terms of **Boolean guard** semantics.

# Guard semantics for our probabilistic language

```
1 | P1 =~ Decision(0.86)
2 | if P1:
3 |     P2 =~ Decision(0.53)
4 |     a = "Paris"
5 |     if P2: return a
6 |     else: raise ValueError(a)
7 | else:
8 |     raise ValueError(P1)
```

- **Question:** What are the necessary conditions for an exception **Exc** to be *reached* or thrown (i.e., the CoT fails)?

# Guard semantics for our probabilistic language

```
1 | P1 =~ Decision(0.86)
2 | if P1:
3 |     P2 =~ Decision(0.53)
4 |     a = "Paris"
5 |     if P2: return a
6 |     else: raise ValueError(a)
7 | else:
8 |     raise ValueError(P1)
```

- **Question:** What are the necessary conditions for an exception **Exc** to be *reached* or thrown (i.e., the CoT fails)?

$$G := \underbrace{\text{Exc} \leftrightarrow \neg P1 \vee (P1 \wedge \neg P2)}_{\text{Boolean guard for Exc}}$$

# Guard semantics for our probabilistic language

```
1 | P1 =~ Decision(0.86)
2 | if P1:
3 |     P2 =~ Decision(0.53)
4 |     a = "Paris"
5 |     if P2: return a
6 |     else: raise ValueError(a)
7 | else:
8 |     raise ValueError(P1)
```

- **Question:** What are the necessary conditions for an exception **Exc** to be *reached* or thrown (i.e., the CoT fails)?

$$G := \underbrace{\text{Exc} \leftrightarrow \neg P1 \vee (P1 \wedge \neg P2)}_{\text{Boolean guard for Exc}}$$

For our simple language,  $\text{WMC}(G \wedge \text{Exc})$  then yields the probability of an exception  $P(\text{Exc})$  (**note:**  $\text{WMC}(G) = 1.$ )

# Boolean expressions in SymPy

```
1 from sympy.logic.boolalg import to_cnf
2 from sympy import symbols, And, Or, Not, Implies
3
4 E, P1, P2 = symbols("E P1 P2")
5 ## biconditional
6 G = And(
7     Implies(E, Or(Not(P1), And(P1, Not(P2)))),
8     Implies(Or(Not(P1), And(P1, Not(P2))), E)
9 )
10 print(G.simplify())
11 ## => (E & ~P1) | (E & ~P2) | (P1 & P2 & ~E)
12 print((G & E).simplify())
13 ## => E & (~P1 | ~P2)
14
15 ## convert to conjunctive normal form (CNF)
16 print(to_cnf(G))
17 ##=> (E | P1) & (E | P2 | ~P1) & (P1 | ~E | ~P1) & (~E | ~P1 | ~P2)
```

- Useful tool for implementing logic, translating formulas into different normal forms, formula simplification.

# Computing semantics via NodeVisitors and sympy

```
1 class ProgramToSemantics(ast.NodeVisitor):
2     def __init__(self):
3         self.stack = [sympy.true]
4         self.guards = {} ## <--- guard formulas
5
6     def visit_If(self, node): ## <--- if-then conds
7         parent = self.stack[-1]
8         cond = extract_cond(node.test) ## <--- Extract Boolean conditions
9         then = sympy.And(self.stack[-1], cond)
10        self.stack.append(then)
11
12        for stmt in node.body: self.visit(stmt)
13        self.stack.pop()
14
15        elseg = sympy.And(parent, sympy.Not(cond))
16        ## else and exception branch
17        if len(node.orelse) == 1:
18            E = sympy.Symbol("Exc")
19            self.guards.setdefault(E, []).append(elseg)
```

- **note:** very minimal implementation, not general; a more complete implementation is left as an exercise.

# Computing semantics via NodeVisitors and sympy

```
1 def extract_cond(test):
2     """Extracts the if conditions in (ast) and translates to sympy.
3
4     Arguments:
5         test: The node containing the if test code.
6     """
7     ## recursively parse boolean expressions
8     if isinstance(test, ast.BoolOp):
9         cond_formulas = [extract_cond(i) for i in test.values]
10        sympy_op = sympy.And if isinstance(test.op, ast.And) else \
11            sympy.Or
12        return sympy_op(*cond_formulas)
13    ## create symbols
14    elif isinstance(test, ast.Name):
15        return sympy.Symbol(test.id)
16    ## negation
17    elif isinstance(test, ast.UnaryOp) and isinstance(test.op, ast.Not):
18        return sympy.Not(extract_cond(test.operand))
19
20    ## example
21    expr = ast.parse("A or B or not (C and D)").body[0].value
22    print(extract_cond(expr))
23    # => A | B | ~(C & D)
```

# Putting it together

```
1 def compile_code(code, func_name):
2     if "def" not in code:
3         P_lines = [" " + l for l in code.split('\n')]
4         code = f"def {func_name}():{'\n'.join(P_lines)}"
5
6     tree = ast.parse(code)
7     tree = ProgramTransformer().visit(tree)
8     ast.fix_missing_locations(tree)
9
10    sem = ProgramToSemantics() ## <-- semantic visitor
11    sem.visit(tree)
12    print(sem.guards) ## print guards
13
14    code = compile(tree, filename="<ast>", mode="exec")
15    locs = {}
16    exec(code, {"Decision" : Decision}, locs)
17
18    return locs[func_name]
19
20 class Compiler:
21     def __call__(self, p):
22         pname = p.__name__
23         program_src = inspect.getsource(p)[11:]
24         compiled = compile_code(program_src, pname)
25         @wraps(p)
26         def wrapper(*args, **kwargs):
27             return compiled(*args, **kwargs)
28         return wrapper
```

## References I